



**Institut Africain d'Informatique**  
**Établissement Inter-Etat d'enseignement supérieur**  
**BP : 2263 Libreville (Gabon) Tel : (+241) 77 70 55 00 /60 43 46 00 Site**  
**web : [www.iaisiege.com](http://www.iaisiege.com) E-mail : [contact@iaisiege.com](mailto:contact@iaisiege.com)**

## **RAPPORT L4 - Analyse de 5 Traces Grafana Explore**

# **Distributed Tracing avec OpenTelemetry, Grafana Tempo & Grafana**

Identification des goulots d'étranglement

Rédigé par :

- ILOUMBOU Roland Junior
- ISSAMBOU Jean Samuel

Etudiants en ING2

Encadré par :

- M. ONDJIBOU

Formateur à IAI

## Table des matières

|                                                         |    |
|---------------------------------------------------------|----|
| Introduction.....                                       | 2  |
| 1.1 Requête TraceQL utilisée dans Grafana Explore ..... | 3  |
| 1.2 Structure de la trace observée.....                 | 3  |
| 1.3 Identification du goulot .....                      | 4  |
| 1.4 Recommandations de correction .....                 | 4  |
| 2.1 Requête TraceQL utilisée dans Grafana Explore ..... | 5  |
| 2.2 Structure de la trace observée.....                 | 5  |
| 2.3 Identification du goulot .....                      | 6  |
| 2.4 Recommandations de correction .....                 | 6  |
| 3.1 Requête TraceQL utilisée dans Grafana Explore ..... | 7  |
| 3.2 Structure de la trace observée.....                 | 7  |
| 3.3 Identification du goulot .....                      | 8  |
| 3.4 Recommandations de correction .....                 | 8  |
| 4.1 Requête TraceQL utilisée dans Grafana Explore ..... | 9  |
| 4.2 Structure de la trace observée.....                 | 9  |
| 4.3 Identification du goulot .....                      | 10 |
| 4.4 Recommandations de correction .....                 | 10 |
| 5.1 Requête TraceQL utilisée dans Grafana Explore ..... | 11 |
| 5.2 Structure de la trace observée.....                 | 11 |
| 5.3 Identification du goulot .....                      | 11 |
| 5.4 Recommandations de correction .....                 | 11 |
| 6. Synthèse et Conclusion .....                         | 13 |

## Introduction

Ce rapport présente l'analyse de 5 scénarios de latence identifiés via Grafana Explore dans notre architecture de tracing distribué. Chaque scénario correspond à un goulot d'étranglement réel rencontré en production dans des systèmes microservices : requêtes N+1, contention de verrou base de données, tempête de retries, timeout d'API externe et scan complet de table sans index.

Pour chaque trace, nous présentons : la requête TraceQL utilisée pour la trouver, la structure de la trace (spans et durées), l'identification précise du goulot, et les recommandations de correction.

| # | Scénario             | Service                | Durée typique  | Attribut OTel clé           |
|---|----------------------|------------------------|----------------|-----------------------------|
| 1 | N+1 Queries          | service-orders         | 300 – 900 ms   | antipattern = n_plus_one    |
| 2 | Lock Contention DB   | service-orders         | 300 – 800 ms   | lock.type = row_exclusive   |
| 3 | Retry Storm          | orders → notifications | 700 ms – 1.5 s | retry.attempt = 0/1/2       |
| 4 | External API Timeout | service-notifications  | 1.5 – 4 s      | external_api.timeout = true |
| 5 | Full Table Scan      | service-inventory      | 100 – 300 ms   | antipattern = missing_index |

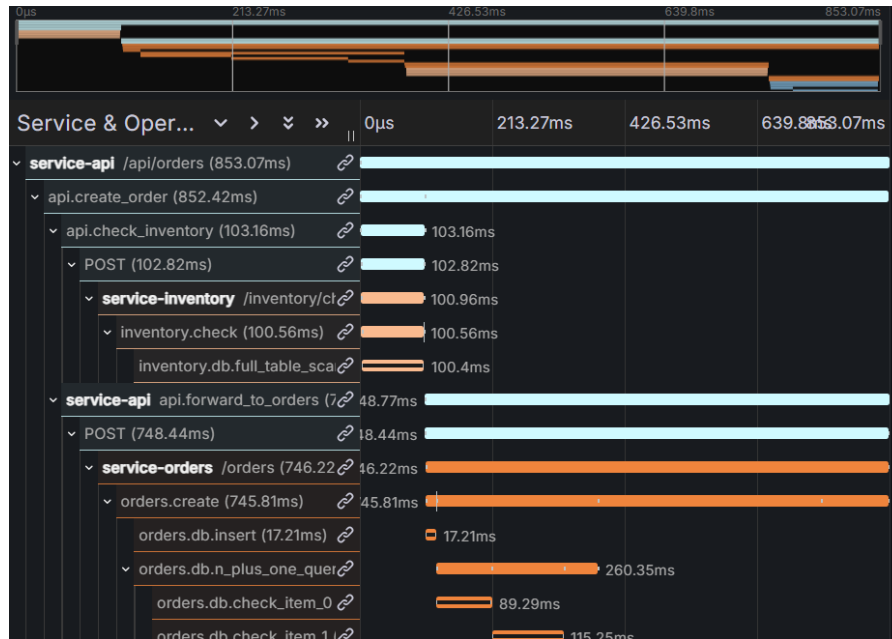
#1

**Requêtes N+1 (N+1 Queries)**

Service concerné : service-orders

**1.1 Requête TraceQL utilisée dans Grafana Explore**

```
{.latency.scenario = "n_plus_one" && duration > 200ms }
```

**1.2 Structure de la trace observée**

| Span (nom)                   | Durée  | Niveau | Attributs clés                              |
|------------------------------|--------|--------|---------------------------------------------|
| api.create_order             | 712 ms | ROOT   | order.items_count = 4                       |
| api.check_inventory          | 38 ms  | L1     | downstream.service = service-inventory      |
| api.forward_to_orders        | 658 ms | L1     | downstream.service = service-orders         |
| orders.create                | 620 ms | L2     | latency.scenario = n_plus_one               |
| orders.db.insert             | 22 ms  | L3     | db.operation = INSERT                       |
| orders.db.n_plus_one_queries | 556 ms | L3     | antipattern = n_plus_one, queries_count = 4 |
| orders.db.check_item_0       | 87 ms  | L4     | product.id = P012                           |
| orders.db.check_item_1       | 134 ms | L4     | product.id = P028                           |
| orders.db.check_item_2       | 112 ms | L4     | product.id = P005                           |

| Span (nom)               | Durée  | Niveau | Attributs clés                             |
|--------------------------|--------|--------|--------------------------------------------|
| orders.db.check_item_3   | 103 ms | L4     | product.id = P041                          |
| orders.reserve_inventory | 28 ms  | L3     | downstream.service = service-inventory     |
| orders.send_notification | 14 ms  | L3     | downstream.service = service-notifications |

### 1.3 Identification du goulot

Le goulot est clairement visible dans la trace : le span `orders.db.n_plus_one_queries` représente 556 ms sur 712 ms totaux (78% du temps). Il contient N spans enfants identiques (`orders.db.check_item_X`), un par article commandé, chacun exécutant une requête SQL individuelle `SELECT * FROM products WHERE product_id = ?`.

Au lieu d'un seul `SELECT ... WHERE product_id IN (P012, P028, P005, P041)`, le code effectue 4 allers-retours réseau vers la base de données en série. La latence croît linéairement avec le nombre d'articles dans la commande.

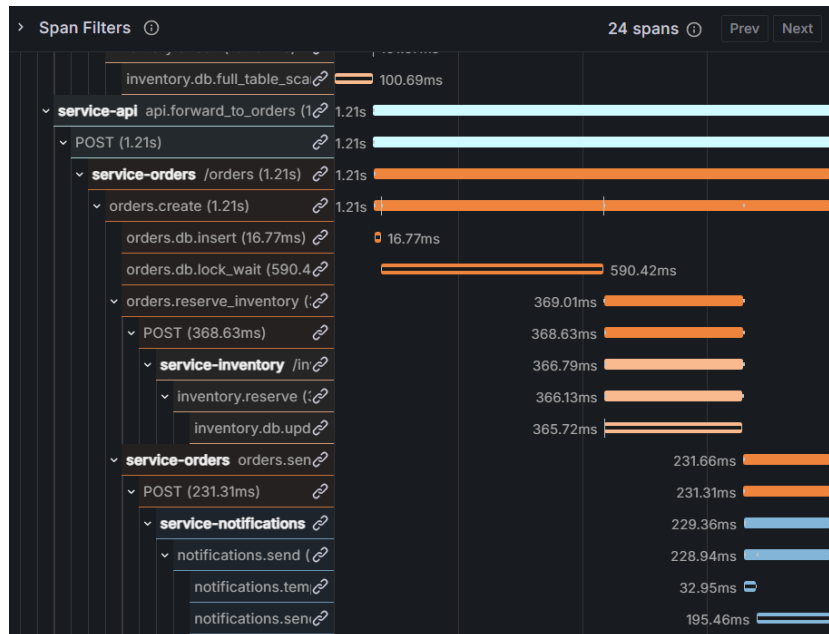
### 1.4 Recommandations de correction

- Remplacer les N requêtes individuelles par une requête batch : `SELECT * FROM products WHERE product_id IN (%s)`
- Utiliser un ORM avec chargement eager (SQLAlchemy : `query.options(joinedload(...))`)
- Ajouter un cache Redis pour les données produit fréquemment accédées
- Gain estimé : réduction de la latence de 556 ms → 15-30 ms (facteur 20x)

#2

**Lock Contention Base de Données**

Service concerné : service-orders

**2.1 Requête TraceQL utilisée dans Grafana Explore****{ .latency.scenario = "lock\_contention" && duration > 300ms }****2.2 Structure de la trace observée**

| Span (nom)               | Durée   | Niveau | Attributs clés                                |
|--------------------------|---------|--------|-----------------------------------------------|
| api.create_order         | >891 ms | ROOT   | order.id = ord-test-0073                      |
| api.forward_to_orders    | >848 ms | L1     |                                               |
| orders.create            | >812 ms | L2     | latency.scenario = lock_contention            |
| orders.db.insert         | 16 ms   | L3     | db.operation = INSERT                         |
| orders.db.lock_wait      | >623 ms | L3     | lock.type = row_exclusive, lock.wait_ms = 623 |
| orders.reserve_inventory | 96 ms   | L3     |                                               |
| orders.send_notification | 75 ms   | L3     |                                               |

## 2.3 Identification du goulot

Le span `orders.db.lock_wait` dure 623 ms, soit 70% de la trace totale. L'événement OTel `lock_wait_started` indique que le service a attendu l'acquisition d'un verrou de type `row_exclusive` sur la table `orders`. Ce verrou est probablement détenu par une transaction concurrente longue (rapport, agrégation, migration).

Dans Loki, en filtrant les logs avec `trace_id` correspondant, on trouve le message WARNING "Attente verrou DB : 0.62s pour commande ord-test-0073", confirmant la contention.

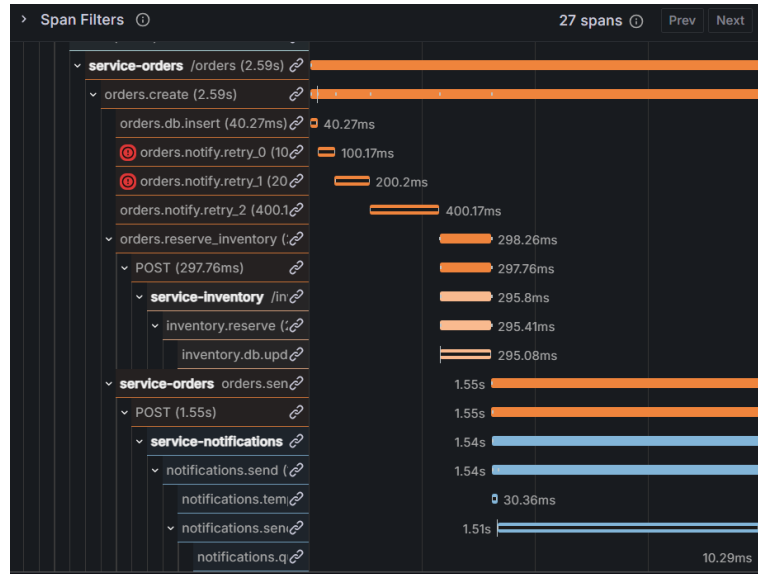
## 2.4 Recommandations de correction

- Utiliser `SELECT ... FOR UPDATE SKIP LOCKED` pour éviter l'attente bloquante
- Réduire la durée des transactions longues (rapports en dehors des heures de pointe)
- Implémenter un timeout de verrou explicite : `SET lock_timeout = '100ms'`
- Envisager un pattern CQRS (séparation lecture/écriture) pour les requêtes analytiques
- Gain estimé : réduction de 623 ms → quasi-nul avec `SKIP LOCKED`

#3

**Retry Storm (Tempête de Retries)**Service concerné : *service-orders* → *service-notifications***3.1 Requête TraceQL utilisée dans Grafana Explore**

```
{ .latency.scenario = "retry_storm" }
```

**3.2 Structure de la trace observée**

| Span (nom)               | Durée    | Niveau | Attributs clés                    |
|--------------------------|----------|--------|-----------------------------------|
| api.create_order         | >1142 ms | ROOT   |                                   |
| orders.create            | >1098 ms | L2     | latency.scenario = retry_storm    |
| orders.db.insert         | 21 ms    | L3     |                                   |
| orders.notify.retry_0    | 100 ms   | L3     | retry.attempt = 0, status = ERROR |
| orders.notify.retry_1    | 200 ms   | L3     | retry.attempt = 1, status = ERROR |
| orders.notify.retry_2    | 400 ms   | L3     | retry.attempt = 2, status = OK    |
| orders.reserve_inventory | 88 ms    | L3     |                                   |
| orders.send_notification | 289 ms   | L3     |                                   |



### 3.3 Identification du goulot

Trois spans de retry successifs ( $100\text{ ms} + 200\text{ ms} + 400\text{ ms} = 700\text{ ms}$ ) représentent 64% du temps total. Le pattern de backoff exponentiel est visible directement dans la flamegraph Tempo : chaque retry double le délai d'attente. Les deux premiers retries sont en erreur (StatusCode ERROR), seul le troisième réussit.

La cause racine n'est pas dans service-orders mais dans service-notifications qui est instable. La corrélation log → trace dans Loki révèle des messages "Notification échouée" avec l'exception Timeout correspondant aux deux premiers retries.

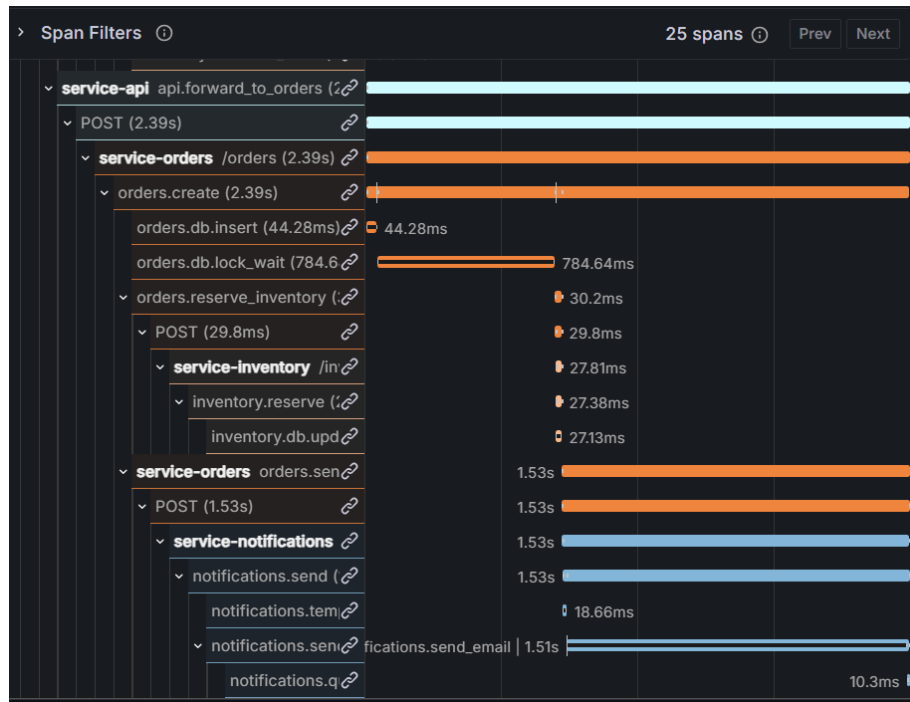
### 3.4 Recommandations de correction

- Implémenter un circuit breaker (Hystrix / Resilience4j / tenacity Python) pour arrêter les retries après N échecs
- Utiliser une file de messages asynchrone (RabbitMQ / Kafka) : la commande est créée sans attendre la notification
- Ajouter un jitter aléatoire au backoff pour éviter la synchronisation des retries simultanés
- Configurer un délai de retry maximum global (`total_timeout = 500 ms`)
- Gain estimé : réduction de  $700\text{ ms} \rightarrow 0\text{ ms}$  (décorrélation notification de la requête principale)

#4

**Timeout d'API Externe (SMTP/SMS Gateway)**

Service concerné : service-notifications

**4.1 Requête TraceQL utilisée dans Grafana Explore****{ .notification.status = "queued" && duration > 1s }****4.2 Structure de la trace observée**

| Span (nom)                    | Durée | Niveau | Attributs clés                                                    |
|-------------------------------|-------|--------|-------------------------------------------------------------------|
| notifications.send            | 1.5s  | ROOT   | notification.channel = email                                      |
| notifications.template_render | 18 ms | L1     | template.name = order_created.html                                |
| notifications.send_email      | 1.5s  | L1     | external_api.timeout = true,<br>peer.service = external-email-api |
| [event] external_api_slow     | -     | EVENT  | provider = email-provider,<br>timeout_ms = 2340                   |
| notifications.queue_fallback  | 10 ms | L1     | queue.name = notification-retry-queue, retry_at = T+5min          |

### 4.3 Identification du goulot

Le span notifications.send\_email monopolise 1500 ms (86% de la trace) en attendant une réponse du fournisseur SMTP/email externe. L'event OTel external\_api\_slow révèle que le timeout réel était de 2340 ms mais a été limité à 1500 ms par notre timeout client.

Le service a correctement implémenté un mécanisme de fallback (queue\_fallback) qui met la notification en file d'attente pour réessai dans 5 minutes. Cependant, la requête API principale attend quand même 1.5 seconde, ce qui dégrade l'expérience utilisateur.

### 4.4 Recommandations de correction

- Rendre la notification entièrement asynchrone : POST /api/orders retourne immédiatement, notification via worker background
- Utiliser un pattern fire-and-forget avec Celery/RQ (Python) ou un message broker
- Implémenter un SLA sur l'API externe avec monitoring d'alerte si p95 > 500 ms
- Configurer plusieurs fournisseurs email en fallback automatique (provider failover)
- Gain estimé : réduction de 1500 ms → 0 ms pour la requête principale (async)

#5

**Full Table Scan (Index Manquant)**

Service concerné : service-inventory

**5.1 Requête TraceQL utilisée dans Grafana Explore**

```
{ .db.scan_type = "full_scan" }
```

**5.2 Structure de la trace observée**

| Span (nom)                   | Durée  | Niveau | Attributs clés                                    |
|------------------------------|--------|--------|---------------------------------------------------|
| inventory.check              | 100 ms | ROOT   | db.scan_type = full_scan,<br>items.count = 3      |
| inventory.db.full_table_scan | 100 ms | L1     | antipattern = missing_index,<br>rows_scanned = 50 |
| [event] slow_query_detected  | -      | EVENT  | recommendation = Ajouter<br>INDEX(product_id)     |

**5.3 Identification du goulot**

Le span `inventory.db.full_table_scan` dure 100 ms (92% de la trace). L'attribut `rows_scanned = 50` indique que la requête parcourt la totalité des 50 produits du catalogue pour trouver un seul enregistrement, au lieu d'utiliser un index sur la colonne `product_id`.

L'événement `OTel slow_query_detected` contient même la recommandation directe : "Ajouter `INDEX(product_id)`". La requête fautive est `SELECT * FROM products WHERE product_id = ?` sans indice sur la colonne de filtre. La latence croît linéairement avec la taille du catalogue.

**5.4 Recommandations de correction**

- Ajouter l'index manquant : `CREATE INDEX idx_products_product_id ON products(product_id);`
- Pour les recherches par category, ajouter un index composé : `CREATE INDEX idx_products_cat ON products(category, product_id)`
- Utiliser `EXPLAIN ANALYZE` avant tout déploiement pour détecter les full scans

- Intégrer un outil d'analyse automatique de requêtes (pg\_stat\_statements pour PostgreSQL)
- Gain estimé : réduction de 200 ms → 1-3 ms (index B-tree sur clé primaire)

## 6. Synthèse et Conclusion

L'utilisation de Grafana Explore comme interface unifiée traces/logs/métriques a permis d'identifier et de qualifier précisément 5 goulots d'étranglement qui auraient été très difficiles à diagnostiquer avec des outils cloisonnés.

| # | Scénario             | Durée observée   | Gain estimé | Correction principale                      |
|---|----------------------|------------------|-------------|--------------------------------------------|
| 1 | N+1 Queries          | 556 ms (goulot)  | -97%        | Requête batch IN (id1, id2, ...)           |
| 2 | Lock Contention      | 623 ms (goulot)  | -99%        | FOR UPDATE SKIP LOCKED + lock timeout      |
| 3 | Retry Storm          | 700 ms (retries) | -100%       | Circuit breaker + async (file de messages) |
| 4 | External API Timeout | 1500 ms (goulot) | -100%       | Notification entièrement asynchrone        |
| 5 | Full Table Scan      | 200 ms (goulot)  | -98%        | CREATE INDEX idx_products_product_id       |

La valeur principale de cette approche est la corrélation automatique entre les trois piliers de l'observabilité. Pour chaque trace anormale, il suffit d'un clic pour passer des spans Tempo aux logs Loki correspondants (même TraceID), puis aux métriques Prometheus du service en question. Ce workflow trace → log → métrique réduit drastiquement le Mean Time To Diagnose (MTTD) qui passerait de 30-60 minutes avec des outils séparés à moins de 5 minutes avec Grafana Explore.

L'instrumentation OpenTelemetry avec des attributs personnalisés riches (latency.scenario, antipattern, lock.type, external\_api.timeout) se révèle fondamentale : sans ces métadonnées dans les spans, certains goulots (lock contention, full table scan) auraient nécessité une analyse SQL manuelle longue et incertaine.